



JavaFX数据绑定应用典型实例

北京理工大学计算机学院
金旭亮

JavaFX数据绑定“编程模式”



绑定数据源
(具备改变通知能力的
数据对象与数据集合)

ObservableList<T>、
ObservableValue、
JavaFX Bean Property



JavaFX数据绑定机制

Binding对象



JavaFX应用程序
的UI界面

包容TableView、Label等支
持数据绑定的控件

示例：利用绑定实现圆的居中显示



AlwaysCenter.java

```
public void start(Stage primaryStage) {  
    Circle circle = new Circle(centerX: 100, centerY: 100, radius: 100);  
    circle.setFill(Color.BLUE);  
    Pane pane = new Pane();  
    pane.getChildren().add(circle);  
  
    Scene scene = new Scene(pane, width: 300, height: 300);  
    //将scene的width属性除以2, 绑定到circle的圆心x坐标上  
    //将scene的height属性除以2, 绑定到circle的圆心y坐标上  
    //让圆始终居于窗体中央  
    circle.centerXProperty().bind(  
        Bindings.divide(scene.widthProperty(), 2));  
    circle.centerYProperty().bind(  
        Bindings.divide(scene.heightProperty(), 2));  
  
    primaryStage.setScene(scene);  
    primaryStage.setTitle("我居中央!");  
    primaryStage.show();  
}
```

此实例告诉我们，JavaFX内置的Bindings工具类提供有一些方法，能进行一些简单的计算和转换，可以应用于数据绑定。

JavaFX控件绑定到数据对象示例



Label控件绑定到MyNumber类里的number属性，点击按钮时，MyNumber对象的number属性值更改，Label控件自动显示其值。

示例代码所在包：mynumber

示例分析

```
public class MyNumber {  
    3 usages  
    private DoubleProperty number;  
    2 usages  
    public final double getNumber(){  
        return numberProperty().get();  
    }  
    2 usages  
    public final void setNumber(double number){  
        this.numberProperty().set(number);  
    }  
    3 usages  
    public final DoubleProperty numberProperty(){  
        if(number==null){  
            number=new SimpleDoubleProperty( initialValue: 0);  
        }  
        return number;  
    }  
}
```

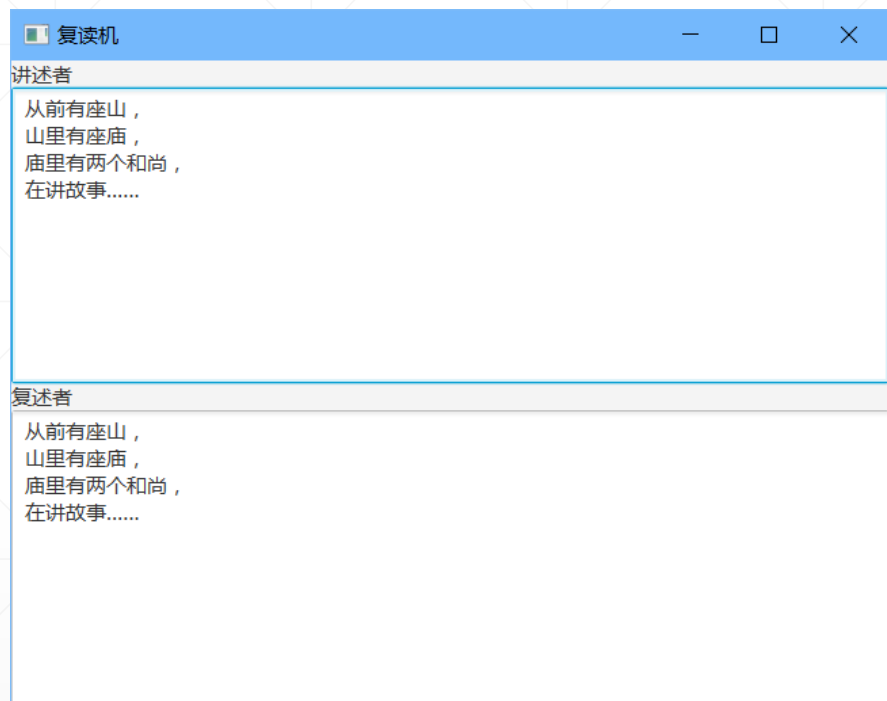


这里面的关键在于使用SDK内置的JavaFX Bean Property封装数据。

示例分析

```
public class MyNumberController implements Initializable {  
    //数据源  
    3 usages  
    final MyNumber myNum = new MyNumber();  
    //UI控件  
    2 usages  
    @FXML  
    private Label lblStatus;  
    @Override  
    public void initialize(URL location, ResourceBundle resources) {  
        //在UI控件与数据源之间建立数据绑定  
        lblStatus.textProperty()  
            .bind(myNum.numberProperty().asString());  
    }  
    1 usage  
    @FXML  
    private void ButtonClick() {  
        //点击按钮, 则修改Number属性值  
        myNum.setNumber(myNum.getNumber() + 1);  
    }  
}
```

“复读机”示例

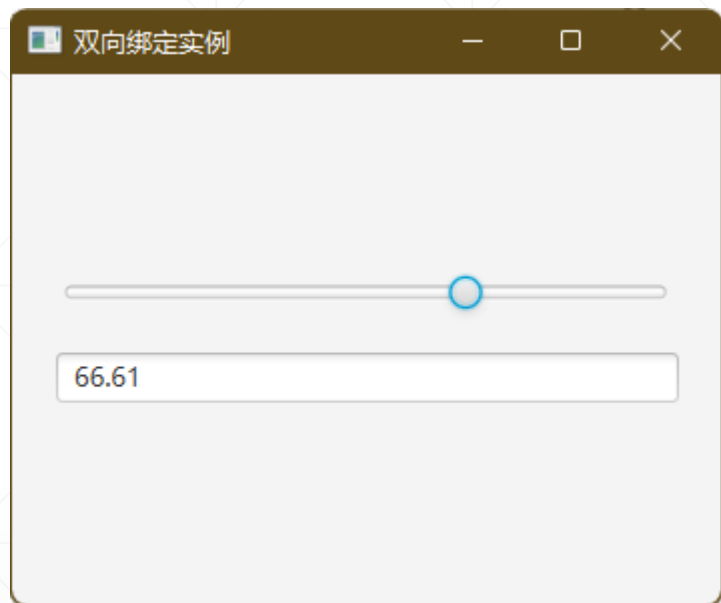


Repeater.java

```
public void start(Stage primaryStage) {  
    TextArea speaker = new TextArea();  
    TextArea repeater = new TextArea();  
    //在两个TextArea控件的text属性中建立双向绑定  
    repeater.textProperty().bindBidirectional(  
        speaker.textProperty());  
    VBox root = new VBox();  
    root.getChildren().addAll(new Label(text: "讲述者"), speaker,  
        new Label(text: "复述者"), repeater);  
    Scene scene = new Scene(root);  
    primaryStage.setScene(scene);  
    primaryStage.setTitle("复读机");  
    primaryStage.show();  
}
```

两个TextArea控件通过textProperty()建立了双向数据绑定。任何一方更改，另一方都会自动地同步。

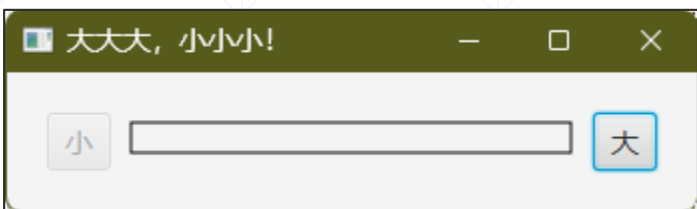
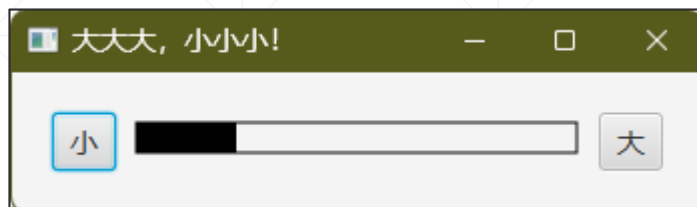
双向绑定示例



```
public class BiDirectionBindController implements Initializable {
    3 usages
    @FXML
    private Slider slider;
    3 usages
    @FXML
    private TextField textField;
    @Override
    public void initialize(URL url, ResourceBundle resourceBundle) {
        slider.setValue(50);
        textField.setText(50 + "");
        textField.textProperty().bindBidirectional(
            slider.valueProperty(), NumberFormat.getInstance());
    }
}
```

示例代码在bidirectionbindtoslider包中

示例：利用数据绑定同步控件的状态



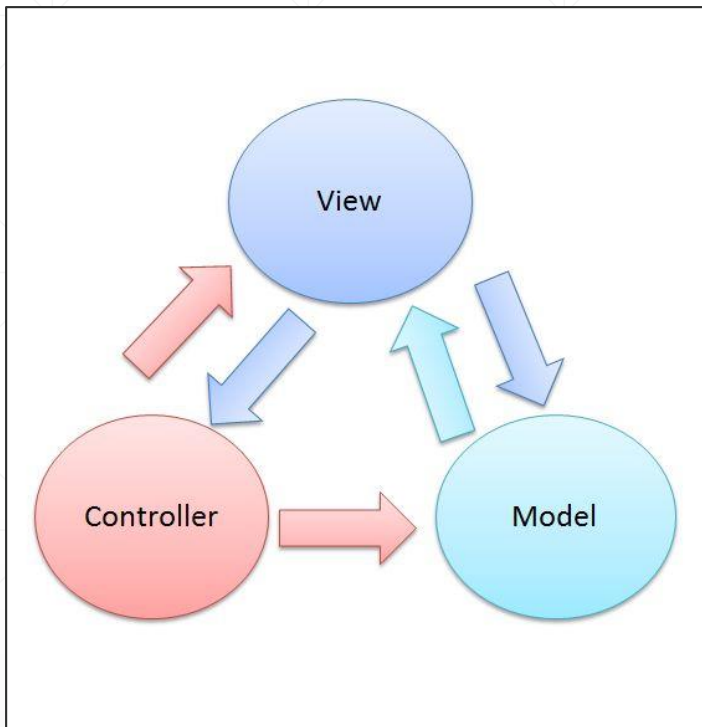
示例：SmallerAndLarger.java

```
import static javafx.beans.binding.Bindings.*;
```

```
Button smaller = new Button( text: "小");
Button larger = new Button( text: "大");
Rectangle gauge = new Rectangle( x: 0, y: 5, width: 50, height: 15);
Rectangle outline = new Rectangle( x: 0, y: 5, width: 220, height: 15);
outline.setFill(null);
outline.setStroke(Color.BLACK);
Pane pane = new Pane();
pane.getChildren().addAll(gauge, outline);

// 点击按钮, 修改Rectangle宽度
smaller.setOnAction(event -> gauge.setWidth(gauge.getWidth() - 10));
larger.setOnAction(event -> gauge.setWidth(gauge.getWidth() + 10));
// 将Rectangle的宽度, 绑定到Button的disable属性上, 从而实现动态的激活或屏蔽
smaller.disableProperty().bind(lessThanOrEqualTo(gauge.widthProperty(), 0));
larger.disableProperty().bind(greaterThanOrEqualTo(gauge.widthProperty(), 220));
```

JavaFX MVC架构中的数据绑定编程套路

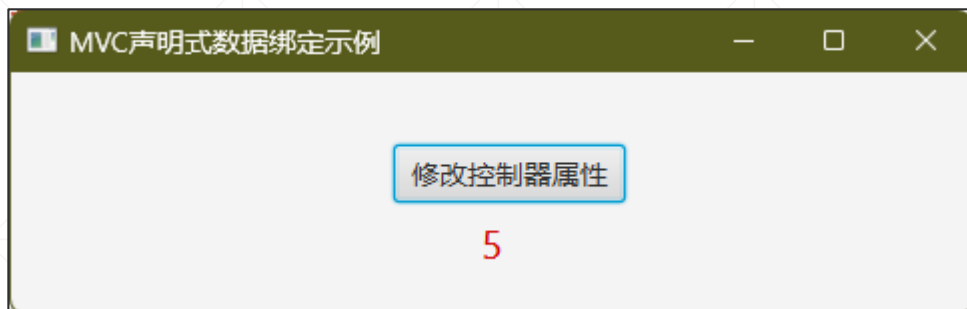


视图由一个FXML文件来定义，每个视图关联着一个**控制器**，在控制器中可以引用在视图中的控件，并且为特定的控件编写事件响应代码。

控制器，封装了Model，它负责更改Model中的属性值。View的刷新可以通过在控制器中监听Model属性值的变化而使用代码实现（前面基本上都是这么干的），也可以直接以“声明”的方式在View中直接定义数据绑定关系，从而实现自动刷新。

“声明式的”数据绑定示例

示例在viewbindtocontroller包中



视图中的数据绑定表达式:

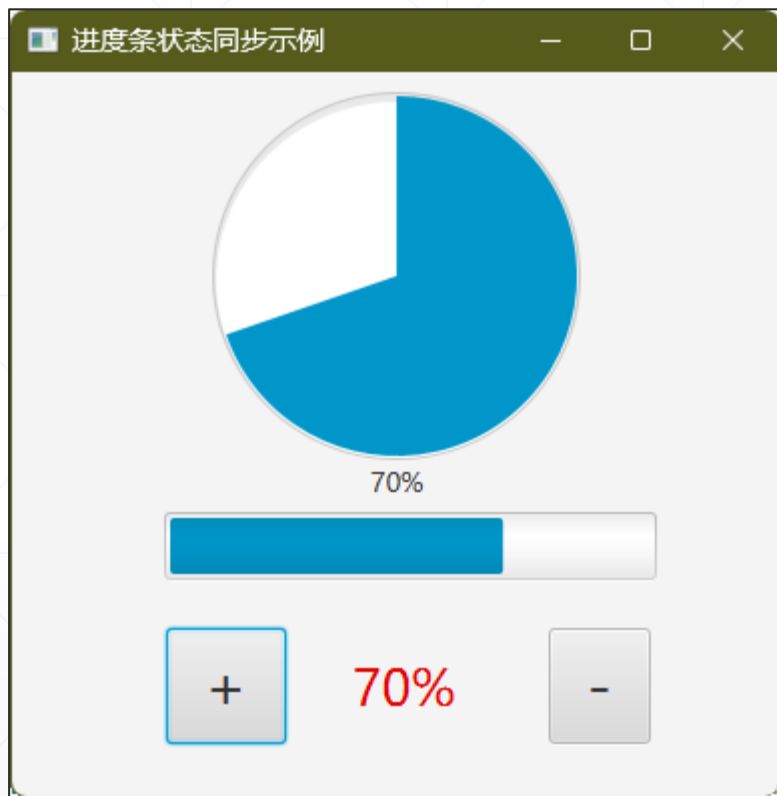
```
<Label text="${controller.counter}" ... >
</Label>
```

技术关键点:

- 控制器中定义了一个counter属性
- View中放置了一个Label控件，它的text属性绑定到了控制器的counter属性上。
- 然后Button的Click事件会修改控制器中的counter属性值。

```
@Override
public void initialize(URL location, ResourceBundle resources) {
    btnClickMe.setOnAction(e -> {
        //直接修改JavaFX Bean属性counter的值
        counter.setValue(counter.getValue() + 1);
    });
}
```

“数据绑定”练习示例



请使用已经掌握的技术，实现左边的示例——点击两个按钮增减值，进度条、饼图和标签同步显示。

参考答案：ProgressDemo示例